

Chapitre 7

LAMP
Apache, MySQL, PHP

Qu'est-ce que la pile LAMP ?

LAMP désigne une architecture de serveur web dynamique basée sur des logiciels libres :

- **L**inux
- **A**pache : serveur HTTP
- **M**ySQL/**M**ariaDB : système de gestion de base de données (MariaDB fork de MySQL)
- **P**HP : langage de script côté serveur

Autres variantes

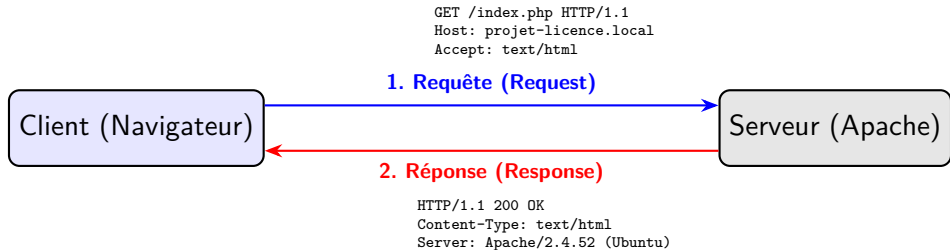
- **WAMP** est similaire à LAMP, mais destinée aux environnements Windows (**W** pour Windows).
- La pile **LEMP** (Linux, **N**ginx, MariaDB, **P**HP) est également très répandue.

Rappel : Le Protocole HTTP

HTTP : HyperText Transfer Protocol

- **Définition** : C'est le protocole de la couche application utilisé pour transférer des ressources (documents HTML, images, API) sur le Web.
- **Modèle** : Fonctionne sur un modèle **Client-Serveur**. Le client (navigateur) initie une connexion, le serveur (Apache) traite et répond.
- **Stateless (Sans état)** : Par défaut, le protocole ne conserve aucune mémoire des requêtes précédentes. Chaque connexion est totalement indépendante. (C'est pourquoi PHP utilise des "Sessions" pour pallier ce manque).

Anatomie d'une connexion HTTP



Les Verbes et Codes de Statut

Pour administrer un serveur web ou lire ses logs d'erreurs (`/var/log/apache2/error.log`), il faut maîtriser la sémantique de base.

Méthodes (Verbes) principales

- **GET** : Demande la représentation d'une ressource (lecture seule).
- **POST** : Soumet des données au serveur (ex : formulaire, modifie l'état).
- **PUT/DELETE** : Remplacement ou suppression (utilisé dans les API REST).

Familles de Codes de Statut

- **2xx (Succès)** : 200 OK.
- **3xx (Redirection)** : 301 Moved Permanently.
- **4xx (Erreur Client)** : 403 Forbidden (Problème de droits!), 404 Not Found.
- **5xx (Erreur Serveur)** : 500 Internal Server Error (Souvent un crash PHP).

Évolution : De HTTP/1.1 à HTTP/3

La Sémantique est Immuable

Les verbes (GET, POST...), les codes de statut (200, 404...) et la structure des en-têtes sont **strictement identiques** sur toutes les versions de HTTP. L'application (PHP) n'a pas besoin d'être réécrite.

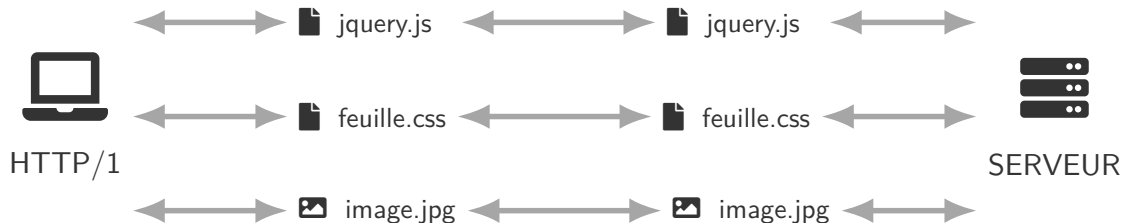
Ce qui évolue, c'est la **couche de transport** (l'optimisation de l'envoi) :

- **HTTP/1.1 (1997)** : Texte brut. Une requête bloque la suivante.
- **HTTP/2 (2015)** : Toujours sur TCP, mais le texte est encodé en **binaire**. Il introduit le **multiplexage** (plusieurs requêtes simultanées dans le même "tuyau").
- **HTTP/3 (2022)** : Abandonne TCP pour le protocole **QUIC (basé sur UDP)**. Plus rapide et résistant aux pertes de réseau (ex : passage du Wi-Fi à la 4G).

Fonctionnement de HTTP/1

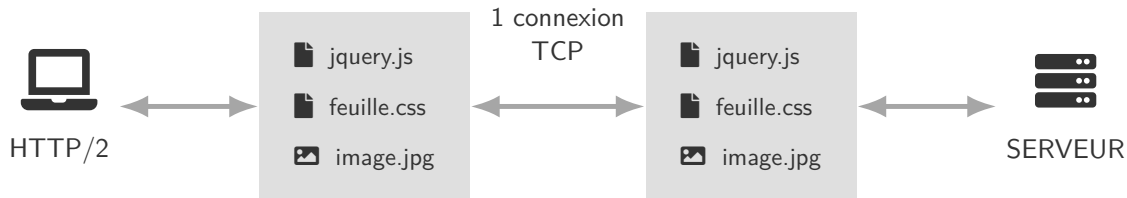
Avec HTTP/1, les navigateurs effectuent une requête par ressource, de plus les ressources sont envoyés au format texte : ce qui entraîne un transfert de données considérable.

3 connexions TCP



Multiplexage HTTP/2

Avec HTTP/2, les navigateurs téléchargent avec une seule requête toutes les ressources nécessaires à l'affichage d'une page, de plus les ressources sont envoyés au format binaire, ce qui améliore les performances.



Requête HTTP

Une requête HTTP est un ensemble de lignes envoyé au serveur par le navigateur. Elle comprend :

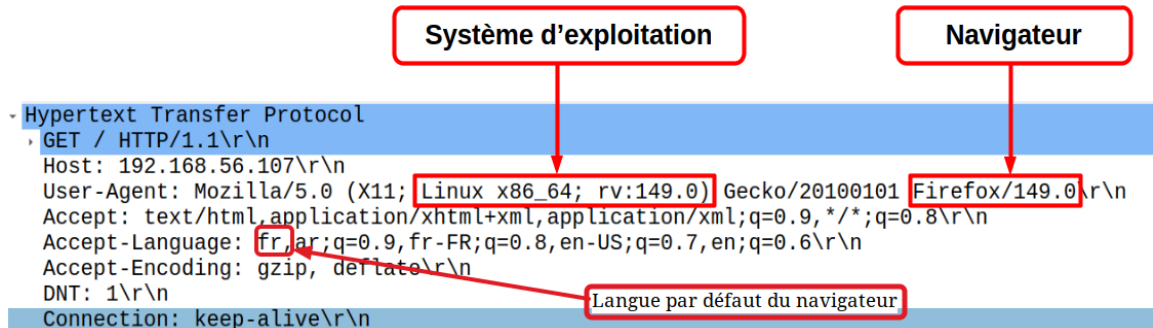
- une ligne de requête précise la méthode qui doit être appliquée, et la version du protocole utilisée. La ligne comprend trois éléments séparés par un espace :
 - la méthode ;
 - l'URL ;
 - la version du protocole utilisé par le client ;

Exemple :

GET / HTTP/1.1

- les champs d'en-tête de la requête : il s'agit d'un ensemble de lignes facultatives permettant de donner des informations supplémentaires sur la requête et/ou le client (Navigateur, système d'exploitation, ...).

Requête HTTP (visualisation avec wireshark)



Réponse HTTP

Une réponse HTTP est un ensemble de lignes envoyées au navigateur par le serveur. Elle comprend :

- ❶ Une ligne de statut composée de trois éléments séparés par un espace :
 - La version du protocole utilisé
 - Le code de statut
 - La signification du code
- ❷ Les champs d'en-tête de la réponse : un ensemble de lignes facultatives permettant de donner des informations supplémentaires sur la réponse et/ou le serveur.
- ❸ Le corps de la réponse : contient le document demandé

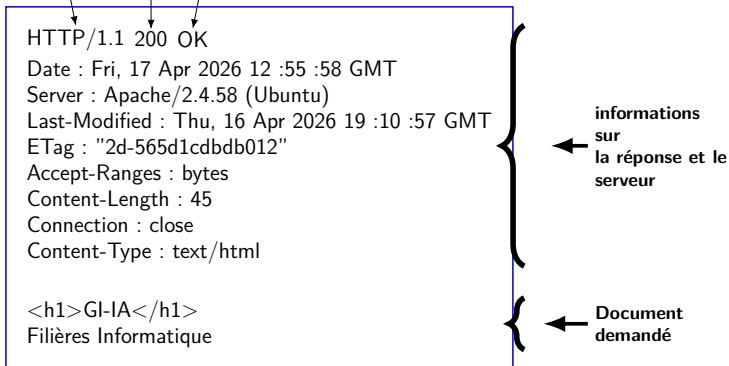
Exemple de requête/réponse HTTP

Requête :

GET / HTTP/1.0

Réponse :

protocole et version utilisé Code du statut Signification du Code



Sémantique HTTP : Propriétés des Méthodes

Rigueur Protocolaire

Le protocole HTTP définit des propriétés fondamentales pour chaque méthode (verbe). Un administrateur doit garantir que le serveur respecte ces principes.

Sécurité (Safe) : La méthode ne doit pas modifier l'état de la ressource sur le serveur (ex : GET, HEAD). Consulter une page ne doit pas supprimer un enregistrement en base de données.

Idempotence : L'exécution répétée d'une même requête doit avoir le même effet que si elle n'avait été exécutée qu'une seule fois (ex : PUT, DELETE).

Erreur classique

Utiliser un lien GET pour supprimer un utilisateur (ex : `/delete.php?id=10`) est une faille de conception majeure.

Anatomie d'un échange : Le rôle du serveur

- **Côté Client** : La requête n'est pas qu'une URL. Elle inclut obligatoirement l'en-tête Host (crucial pour les Virtual Hosts).
- **Côté Serveur** : Le processus de réponse n'est pas atomique.

Flux de traitement interne (Apache)

- ➊ **Réception** : Analyse des en-têtes (Headers).
- ➋ **Mappage** : Traduction de l'URL en chemin système (*DocumentRoot*).
- ➌ **Délégation** : Si fichier `.php`, transmission à l'interpréteur (PHP-FPM).
- ➍ **Assemblage** : Construction de la réponse (Ligne de statut + Headers + Body).

Note : La distinction entre l'en-tête (métadonnées) et le corps (données) est la base du diagnostic dans les logs.

Apache

Installation d'Apache 2.4

```
# Mise à jour des dépôts
sudo apt update
# Installation du paquet
sudo apt install apache2
```

```
# Vérification du statut
sudo systemctl status apache2
# Activation au démarrage
sudo systemctl enable apache2
```

Validation de la configuration

Avant tout redémarrage, toujours tester la syntaxe : `apache2ctl configtest`
Le message Syntax OK confirme l'absence d'erreur.

Organisation des fichiers sous Ubuntu

- `/etc/apache2/` : configuration globale
 - **apache2.conf** : fichier principal
 - **ports.conf** : ports d'écoute (80, 443)
 - **envvars** : variables d'environnement propres à apache
 - **sites-available/** : définitions des sites virtuels
 - **sites-enabled/** : liens symboliques vers les sites actifs
 - **mods-available/** : fichiers de configuration de tous les modules installés
 - **mods-enabled/** : liens symboliques vers les modules actifs
- `/var/www/html/` : racine par défaut des sites
- `/var/log/apache2/` : journaux (`access.log`, `error.log`)
- `/usr/lib/apache2/modules/` : modules chargés

Création d'un site virtuel pour info.ump.ma (HTTP seul)

Fichier /etc/apache2/sites-available/info.ump.ma.conf :

```
<VirtualHost *:80>
    ServerName info.ump.ma
    ServerAlias www.info.ump.ma
    DocumentRoot /var/www/info.ump.ma

    ErrorLog ${APACHE_LOG_DIR}/info_error.log
    CustomLog ${APACHE_LOG_DIR}/info_access.log combined

    <Directory /var/www/info.ump.ma>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Identification et Racine

- **VirtualHost * :80** : Définit le périmètre de la configuration pour toute adresse IP sur le port 80 (HTTP).
- **ServerName** : Définit l'identifiant principal du site. C'est l'URL que le client saisit.
- **ServerAlias** : Permet de définir des noms alternatifs, comme le sous-domaine *www*.
- **DocumentRoot** : Spécifie le chemin absolu vers le répertoire contenant les fichiers sources du site.

Gestion des Journaux Apache sépare les logs pour faciliter le débogage :

- **ErrorLog** : Enregistre les erreurs de diagnostic.
- **CustomLog** : Enregistre les requêtes entrantes. Le format **combined** inclut des détails comme le navigateur utilisé.

Sécurité du Répertoire Le bloc `<Directory>` applique des règles spécifiques au système de fichiers :

- **Options -Indexes** : Si aucun fichier **index.html** n'est présent, Apache n'affichera pas la liste des fichiers du dossier aux visiteurs.
- **AllowOverride All** : Autorise l'utilisation de fichiers **.htaccess** dans le dossier du site pour modifier la configuration localement (souvent requis par WordPress, PrestaShop, etc.).
- **Require all granted** : Accorde l'accès public au contenu.

Gestion des droits d'accès

```
# Création du répertoire du site
sudo mkdir -p /var/www/info.ump.ma
# Attribution des droits à l'utilisateur www-data
sudo chown -R www-data:www-data /var/www/info.ump.ma
# Droits standards : 755 pour dossiers, 644 pour fichiers
sudo chmod -R u=rwX,go=rX /var/www/info.ump.ma
```

```
# Activation du site et rechargement
sudo a2ensite info.ump.ma
sudo systemctl reload apache2
```

Test local

Pour tester avant d'avoir un nom de domaine, ajoutez dans /etc/hosts de la machine cliente :

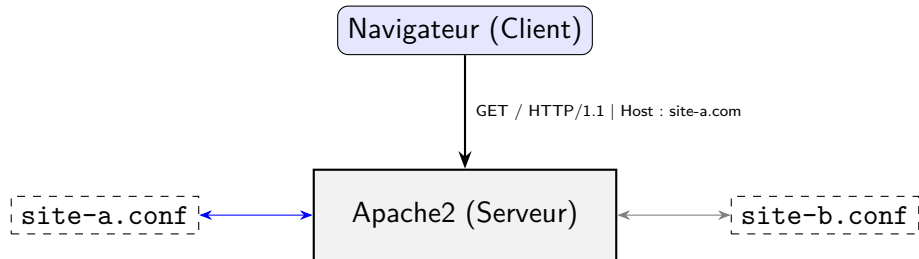
```
# Remplacer 192.168.56.107 par l'adresse IP du serveur Ubuntu
192.168.56.107    info.ump.ma
```

Héberger plusieurs sites sur un même serveur

- Apache peut servir simultanément plusieurs sites web distincts.
- Chaque site possède son propre **Virtual Host** :
 - Nom de domaine différent (ex : `site1.ma`, `site2.com`)
 - Répertoire racine (`DocumentRoot`) séparé
 - Journaux d'accès et d'erreurs indépendants
 - Paramètres spécifiques (PHP, sécurité, etc.)
- Utile pour :
 - Héberger des clients distincts sur un même machine
 - Développer plusieurs projets en parallèle
 - Tester différentes configurations

Héberger plusieurs sites : Mécanisme

- **Problématique** : Comment un serveur avec une seule adresse IP peut-il servir *site-a.com* et *site-b.com* ?
- **Solution** : Apache utilise l'en-tête HTTP Host envoyé par le navigateur.



Workflow de déploiement multi-sites

Pour chaque nouveau site sur le serveur Ubuntu, l'administrateur doit suivre ce cycle :

- ❶ **Arborescence** : Créer un dossier distinct dans `/var/www/`.
- ❷ **Configuration** : Créer un nouveau fichier `.conf` dans `sites-available/`.
- ❸ **Validation** : Vérifier la syntaxe avant de recharger.
- ❹ **Activation** : Créer le lien symbolique vers `sites-enabled/`.

```
# Vérifier la syntaxe (Indispensable en production)
apache2ctl configtest

# Activer le site A
sudo a2ensite site-a.conf

# Rechargement à chaud (sans coupure de service)
sudo systemctl reload apache2
```

Ajout d'un deuxième site : math.ump.ma

- 1 Créer le répertoire racine :

```
sudo mkdir -p /var/www/math.ump.ma  
sudo chown -R www-data:www-data /var/www/math.ump.ma
```

- 2 Créer le fichier de configuration : /etc/apache2/sites-available/math.ump.ma.conf

```
<VirtualHost *:80>  
    ServerName math.ump.ma  
    ServerAlias www.math.ump.ma  
    DocumentRoot /var/www/math.ump.ma  
  
    ErrorLog ${APACHE_LOG_DIR}/math_error.log  
    CustomLog ${APACHE_LOG_DIR}/math_access.log combined  
  
    <Directory /var/www/math.ump.ma>  
        Options -Indexes +FollowSymLinks  
        AllowOverride All  
        Require all granted  
    </Directory>  
</VirtualHost>
```

Activation et gestion des sites

```
# Activer le nouveau site
sudo a2ensite math.ump.ma
# Recharger Apache
sudo systemctl reload apache2
```

```
# Désactiver un site sans supprimer sa configuration
sudo a2dissite math.ump.ma
sudo systemctl reload apache2
```

```
# Lister les sites actifs (liens dans sites-enabled/)
ls -l /etc/apache2/sites-enabled/
```

Test local avec /etc/hosts

Ajouter les deux entrées pour tester :

192.168.56.107	info.ump.ma	www.info.ump.ma
192.168.56.107	math.ump.ma	www.math.ump.ma

Bonnes pratiques pour la gestion multi-sites

- **Nommage des fichiers** : utiliser le nom de domaine complet (`exemple.ma.conf`) pour éviter les confusions.
- **Journaux séparés** : chaque Virtual Host doit avoir ses propres fichiers de logs.
- **Permissions isolées** : chaque site peut avoir un propriétaire différent (ex : `www-data` pour la lecture, mais écriture limitée).
- **PHP-FPM** : possibilité d'utiliser un pool distinct par site (versions PHP différentes, ressources limitées).
- **Sécurité** : éviter de mélanger des sites de confiance différente sur un même serveur sans isolation forte (conteneurs, etc.).

Sites virtuels par adresse IP

Dans cet exemple, nous allons configurer un nouveau site virtuel **phys.ump.ma**, qui utilise une adresse IP différente.

Dans cet exemple, la machine doit être munie, soit de plusieurs interfaces réseaux soit de plusieurs adresses IP associées à la même interface réseau (on parle d'IP aliasing).

IP aliasing

Pour affecter d'autres adresses IP à une interface réseau, il faut ajouter au fichier **/etc/netplan/50-cloud-init.yaml** les lignes suivantes :

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8: # Interface "Privé hôte"
      dhcp4: true # @IP dynamique
      addresses:
        - 192.168.56.201/24 # @IP alias
        - 192.168.56.202/24
        - 192.168.56.203/24
```

Configuration du site virtuel

Il faut créer le répertoire `/var/www/phys.ump.ma`.

Dans `/etc/apache2/sites-available/`, il faut créer le fichier : **phys.ump.ma.conf**

accessible via l'adresse **192.168.56.201**



```
<VirtualHost 192.168.56.201:80>
    DocumentRoot /var/www/phys.ump.ma
    ServerName    phys.ump.ma
    ServerAlias   www.phys.ump.ma

    ErrorLog ${APACHE_LOG_DIR}/phys_error.log
    CustomLog ${APACHE_LOG_DIR}/phys_access.log combined

    <Directory /var/www/phys.ump.ma>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Activation du nouveau site

Il faut activer le site en tapant la commande :

```
sudo a2ensite phys.ump.ma
```

Après l'activation, il faut recharger le serveur apache en tapant la commande

```
sudo systemctl reload apache2
```

Dans la machine cliente, il faut ajouter au fichier **/etc/hosts**, la ligne suivante :

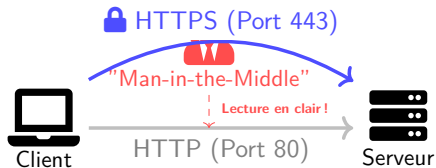
```
192.168.56.201    phys.ump.ma      www.phys.ump.ma
```

Le nouveau site sera accessible via les liens :

<http://phys.ump.ma> ou <http://www.phys.ump.ma>

Sécuriser Apache

- **Confidentialité** : Empêcher l'interception des données sensibles (mots de passe, données personnelles) par un tiers.
- **Intégrité** : Garantir que les données envoyées par le serveur n'ont pas été modifiées durant le transfert.
- **Authentification** : Prouver au visiteur qu'il est bien sur le site officiel et non une copie frauduleuse.



Le protocole TLS/SSL

La sécurisation repose sur l'ajout d'une couche de chiffrement entre la couche Transport (TCP) et la couche Application (HTTP).

Génération du certificat Auto-signé

On utilise **OpenSSL** pour créer une paire de clés (publique/privée).

Commande OpenSSL

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \  
-keyout /etc/ssl/private/apache.key \  
-out /etc/ssl/certs/apache.crt
```

- **-x509** : définit un certificat auto-signé.
- **apache.key** : La clé privée (À GARDER SECRÈTE).
- **apache.crt** : Le certificat (Clé publique).

Génération du certificat Auto-signé

```
sudo openssl req -x509 -nodes -days 365 ...  
  
...  
  
Country Name (2 letter code) [AU]:MA  
State or Province Name (full name) [Some-State]:Oujda  
Locality Name (eg, city) []:Oujda  
Organization Name (eg, company) [Internet Widgits Pty Ltd]:fso  
Organizational Unit Name (eg, section) []:ump  
Common Name (e.g. server FQDN or YOUR name) []:info.ump.ma  
Email Address []:info@ump.ma
```

Vérification du pare-feu

```
# Vérifiez le statut  
sudo ufw status  
  
# Autorisez le trafic HTTP et HTTPS pour Apache.  
sudo ufw allow 'Apache Full'
```

Configuration du VirtualHost 443

Créer le fichier `/etc/sites-available/info.ump.ma-ssl.conf` pour le port HTTPS (443).

```
<VirtualHost *:443>
    ServerName info.ump.ma
    DocumentRoot /var/www/info.ump.ma

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/apache.crt
    SSLCertificateKeyFile /etc/ssl/private/apache.key
</VirtualHost>
```

Activation

```
sudo a2enmod ssl
sudo a2ensite info.ump.ma-ssl
sudo systemctl restart apache2
```

Vérification : <https://info.ump.ma>

Activation de HTTP/2

Prérequis architectural

HTTP/2 **exige** l'utilisation du module `mpm_event` d'Apache. Il est techniquement incompatible avec l'ancien module `mpm_prefork` (et donc avec `Mod_PHP`). C'est une raison supplémentaire d'exiger **PHP-FPM**.

```
# 1. Activer le module HTTP/2 d'Apache
```

```
sudo a2enmod http2
```

```
# 2. Recharger le service pour appliquer le changement
```

```
sudo systemctl restart apache2
```

Activation de HTTP/2 sous Ubuntu

Note : Sur les navigateurs modernes, HTTP/2 et HTTP/3 exigent obligatoirement que le site soit chiffré via HTTPS (TLS).

MariaDB

MariaDB ou MySQL ?

- MariaDB est un fork communautaire de MySQL, créé par les fondateurs originaux.
- Il est **100% compatible** avec les API MySQL.

Installation de MariaDB

```
sudo apt update && sudo apt install mariadb-server  
sudo systemctl enable mariadb  
sudo systemctl start mariadb
```

Vérification

```
sudo systemctl status mariadb
```

Accès initial (root sans mot de passe via socket Unix)

```
sudo mysql
```


Création d'une base et d'un utilisateur dédié

```
-- Création de la BD gi_ia (si elle n'existe pas déjà)
CREATE DATABASE IF NOT EXISTS gi_ia CHARACTER SET utf8mb4 COLLATE
    utf8mb4_unicode_ci;

-- Création de l'utilisateur "info" (s'il n'existe pas déjà)
CREATE USER IF NOT EXISTS 'info'@'localhost' IDENTIFIED BY 'gi_ia_2026';

-- Donner tous les privilèges à "info" sur la BD "gi_ia"
GRANT ALL PRIVILEGES ON gi_ia.* TO 'info'@'localhost';

-- Appliquer les nouveaux privilèges
FLUSH PRIVILEGES;
```

Principe du moindre privilège

Ne jamais utiliser le compte root pour une application web. Créer un utilisateur avec uniquement les droits nécessaires sur la base concernée.

Création d'une table et insertion des données

```
USE gi_ia;

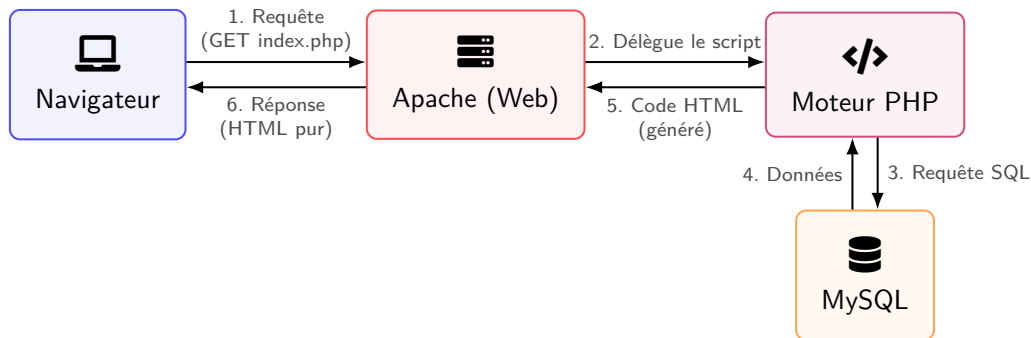
-- Création de la table Etudiant (si elle n'existe pas déjà)
CREATE TABLE IF NOT EXISTS Etudiant (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(20) NOT NULL,
    prenom VARCHAR(20) NOT NULL,
    cne VARCHAR(10) UNIQUE NOT NULL
);

-- Insertion des données
INSERT INTO Etudiant (nom, prenom, cne) VALUES
('Oujdi', 'Anas', '123456789'),
('Berkani', 'Kenza', '987654321'),
('Fassi', 'Yassine', '135792468'),
('Missouri', 'Zineb', '246813579'),
('Tazi', 'Mehdi', '112233445');
```

Php

Introduction à PHP

- **PHP** (*PHP : Hypertext Preprocessor*) est un langage de script exécuté **côté serveur**.
- **Son rôle** : Générer du contenu dynamique et faire le pont avec la base de données.
- **La règle d'or** : Le code source PHP n'est jamais envoyé au navigateur du client. Seul le résultat (du HTML pur) est transmis.



PHP-FPM : la référence moderne

- libapache2-mod-php exécute PHP dans le processus Apache.
- **Inconvénients :**
 - Un processus Apache = une instance PHP (consommation mémoire)
 - Pas d'isolation entre sites (problèmes de sécurité)
 - Pas de mise à l'échelle fine
- **PHP-FPM** (FastCGI Process Manager) :
 - Gère des pools de processus PHP indépendants
 - Communication via socket Unix (performant) ou TCP
 - Permet d'avoir plusieurs versions de PHP en parallèle

Installation de PHP 8.3 FPM

```
sudo apt install php8.3-fpm php8.3-mysql php8.3-cli \  
                php8.3-curl php8.3-gd php8.3-mbstring \  
                php8.3-xml php8.3-zip
```

```
# Vérifier la version installée
```

```
php -v
```

```
# Vérifier le statut
```

```
sudo systemctl status php8.3-fpm
```

Liaison Apache – PHP-FPM

Activer les modules et la configuration :

```
sudo a2enmod proxy_fcgi setenvif  
sudo a2enconf php8.3-fpm  
sudo systemctl reload apache2
```

Que fait `a2enconf php8.3-fpm` ?

Il active un fichier de configuration qui associe les fichiers `.php` au gestionnaire FastCGI via la socket Unix `/run/php/php8.3-fpm.sock`.

```
# Vérifier que la socket existe et appartient à www-data  
ls -l /run/php/php8.3-fpm.sock
```

Personnalisation de la configuration PHP

Fichier principal : `/etc/php/8.3/fpm/php.ini`

```
# Paramètres à adapter selon l'application
upload_max_filesize = 20M
post_max_size = 21M
memory_limit = 256M
max_execution_time = 60

# En développement uniquement :
display_errors = On

# En production :
display_errors = Off
log_errors = On
```

Après modification, redémarrer PHP-FPM : `sudo systemctl restart php8.3-fpm`

Connexion PDO (avec logging)

```
<?php
// 1. Paramètres de connexion
$host = 'localhost';
$dbname = 'gi_ia';
$username = 'info';
$password = 'gi_ia_2026';

try {
    // 2. Connexion à la base de données
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username,
        $password);
    // Activer l'affichage des erreurs de la base de données
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // 3. Préparation et exécution de la requête
    $sql = "SELECT id, nom, prenom, cne FROM Etudiant";
    $stmt = $pdo->query($sql);
```

Affichage

```
// 4. Affichage des résultats
echo "<h1>Liste des Étudiants</h1>";
echo "<ul>";

// Parcourir chaque ligne récupérée
while ($etudiant = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo "<li>";
    echo "<strong>" . htmlspecialchars($etudiant['nom']) . " " .
        htmlspecialchars($etudiant['prenom']) . "</strong> ";
    echo "(CNE: " . htmlspecialchars($etudiant['cne']) . ")";
    echo "</li>";
}

echo "</ul>";

} catch (PDOException $e) {
    // En cas d'erreur de connexion ou de requête
    echo "Erreur de connexion : " . $e->getMessage();
}
?>
```

Configuration du pare-feu UFW

```
# Autoriser HTTP et HTTPS
sudo ufw allow 'Apache Full'

# Restreindre l'accès à MariaDB à localhost uniquement
sudo ufw allow proto tcp from 127.0.0.1 to any port 3306

# Vérification concise
sudo ufw status numbered
```

Vérification

Assurez-vous que le port 3306 n'est pas ouvert en entrée depuis l'extérieur.

Où chercher les erreurs ?

- **Apache** : `/var/log/apache2/error.log`
- **PHP-FPM** : `/var/log/php8.3-fpm.log`
- **MariaDB** : `/var/log/mysql/error.log`
- **Journal système** : `sudo journalctl -xe`

```
# Suivre en temps réel
```

```
sudo tail -f /var/log/apache2/error.log
```

```
# Filtrer les erreurs PHP
```

```
sudo grep "PHP" /var/log/apache2/error.log
```

Conclusion : L'écosystème LAMP

- **Standard robuste** : Une pile technologique open-source qui propulse une grande partie du web mondial depuis plus de 20 ans.
- **Modularité** : Chaque composant est indépendant mais conçu pour s'interfacer parfaitement avec les autres.
- **Communauté** : Une documentation vaste et une résolution de problèmes facilitée par des millions d'utilisateurs.
- **Évolution** : S'adapte en permanence aux standards modernes (HTTP/2, SSL/TLS, MariaDB).

Références & Webographie

- Apache HTTP Server :
<https://httpd.apache.org/docs/>
- Ubuntu Server Guide :
<https://ubuntu.com/server/docs>
- PHP Manual :
<https://www.php.net/manual/fr/>
- Support Apache des serveurs virtuels par nom :
<https://httpd.apache.org/docs/2.4/vhosts/name-based.html>
- Configuration PHP-FPM :
<https://www.php.net/manual/fr/install.fpm.configuration.php>
- MariaDB :
<https://mariadb.com/kb/>